

Group 67

Task 1. Study the Data

TripData

- **What is the primary entity represented by the dataset?** Individual taxi trips.
- **Which attributes uniquely identify a record (i.e., what is the primary key of the dataset)?** `tpep_pickup_datetime` and `tpep_dropoff_datetime`. There are duplicates of these, all of which have a positive or negative sum (hinting at a reimbursement from the company). It may be necessary to create a surrogate key (a new key based on a hash).
- **Which attributes are likely to be used for joins?** `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `PULocationID`, and `DOLocationID`. The time and date are used to join with the Weather and Taxi Zone Lookup datasets.
- **Which attributes are temporal?** `tpep_pickup_datetime` and `tpep_dropoff_datetime`.
- **Which attributes contain categorical values?** `VendorID`, `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `PULocationID`, `DOLocationID`, and `payment_type`.
- **Which attributes are likely to grow over time?** None; the database grows vertically, not horizontally.

Weather

- **What is the primary entity represented by the dataset?** Hourly weather conditions (snow, wind, rain, temperature).
- **Which attributes uniquely identify a record (i.e., what is the primary key of the dataset)?** `year`, `month`, `day`, and `hour`.
- **Which attributes are likely to be used for joins?** `year`, `month`, `day`, and `hour`.
- **Which attributes are temporal?** `year`, `month`, `day`, and `hour`.
- **Which attributes contain categorical values?** `temp_source`, `rhum`, `rhum_source`, `prcp_source`, `snwd_source`, `wdir`, `wdir_source`, `wspd_source`, `wpgt_source`, `pres_source`, `cldc`, `cldc_source`, `coco`, and `coco_source`.
- **Which attributes are likely to grow over time?** None; the database grows vertically, not horizontally.

Air Quality

- **What is the primary entity represented by the dataset?** Air quality based on location and time.
- **Which attributes uniquely identify a record (i.e., what is the primary key of the dataset)?** `State Code`, `County Code`, `Parameter Code`, `Site Num`, `Date GMT`, `Time GMT`, `Method Name`, and `POC`.
- **Which attributes are likely to be used for joins?** `State Code`, `County Code`, `Date GMT`, and `Time GMT`.
- **Which attributes are temporal?** `Date Local`, `Time Local`, `Date GMT`, and `Time GMT`.
- **Which attributes contain categorical values?** `State Code`, `County Code`, `Parameter Code`, `Site`

Num, POC, Datum, Parameter Name, Units of Measure, MDL, Method Type, Method Code, Method Name, State Name, and County Name.

- **Which attributes are likely to grow over time?** None; the database grows vertically, not horizontally.

Taxi Zone Lookup

- **What is the primary entity represented by the dataset?** Locations of taxi zones.
- **Which attributes uniquely identify a record (i.e., what is the primary key of the dataset)?** LocationID.
- **Which attributes are likely to be used for joins?** Borough and Zone.
- **Which attributes are temporal?** N/A
- **Which attributes contain categorical values?** Borough, Zone, and service_zone.
- **Which attributes are likely to grow over time?** N/A

Task 2. Design Your Storage Architecture

Storage Architecture Design

1. Directory Structure

- data/ : Raw, unprocessed source files (CSV/Parquet), read directly by the ingestion framework.
- output_data/ : Processed Delta tables written by execute_pipeline, one subdirectory per dataset.

2. Delta Table Organization & Naming Conventions

- *Naming Convention:* All tables and columns must be lowercase, with words separated by underscores (_).
- yellow_tripdata_2024-01, 2024-02, 2024-03 (unioned) → trip_data
- taxi_zone_lookup.csv → taxi_zones
- air_quality.csv → air_quality
- weather.csv → weather
- Task 5's enriched output → integrated_taxi_trips

3. Partitioning Strategy

- trip_data : Partition by date (year and month).
- weather : Partition by year, month, and day.
- air_quality : Not partitioned — treated as a lookup table (see Design Decisions below).
- taxi_zones : Not partitioned. It is too small (~265 rows) and there is no key worth partitioning on.
- integrated_taxi_trips : Partition by year and month, matching trip_data since it inherits one row per trip.

Design Decisions

Which datasets should conceptually be treated as lookup tables? The taxi_zones and air_quality

datasets are treated as lookup tables. `taxi_zones` is a small, static dimension table (~265 rows).

`air_quality` becomes a lookup table after ingestion filtering: the raw EPA extract is nationwide, but the taxi trips are NYC-only, so `air_quality_scope` filters it to the New York City boroughs (`state_code = 36`, `county_code` in 005/047/081). What remains is 51,885 rows from 6 stations carrying a single pollutant (`parameter_code = 88101`, PM2.5) — small and low-cardinality enough to behave as a lookup table. `weather` is not a lookup table; it is a time-series fact table, which is why it is partitioned by date below.

Which datasets should not be partitioned? Explain why. The `taxi_zones` and `air_quality` datasets should not be partitioned. `taxi_zones` is too small, and partitioning it would make Spark slower (opening many folders to find few items). `air_quality`, once filtered to the NYC boroughs, is only ~52K rows across 3 counties, 6 sites, and 1 pollutant — small enough that any partitioning would produce many tiny files for no scan-reduction benefit. Note that this reasoning depends on the filter: the *raw* nationwide extract would be a genuine candidate for partitioning by `state_code / county_code`, which is exactly the 20× scenario discussed below.

Which datasets require different partitioning strategies?

- `trip_data` and `integrated_taxi_trips` are partitioned temporally by year and month (although this has to be tested).
- `weather` is partitioned temporally as well, but one level finer (year, month, day), since it is a much smaller table where a per-day partition still holds a reasonable number of rows.
- `air_quality` and `taxi_zones` are left unpartitioned (see above).

Under what conditions does partitioning become harmful? Partitioning becomes harmful when there are too many unique values in the partition column. There must be a balance between the number of folders generated and the amount of data inside each folder. Task 6 measures this directly (see `Benchmark Report.md`): the deciding factor is not the partition count but the resulting file size. Partitioning `trip_data` by day yields 278 files of 1.65 MB and is still slightly faster than monthly partitioning, whereas the same year, month, day scheme applied to `weather` yields 366 files averaging 6.6 KB for 2.4 MB of data — a table where per-file overhead dominates and the partitioning should be coarsened. Partitioning is also useless, though not actively harmful, when the workload never filters on the partition column: none of the three benchmark queries does, so neither scheme prunes a single file.

If the total data volume increased by 20×, what changes would you make to your storage design?

Make the partitioning strategy more in depth

- Partition `trip_data`: use the day, not just year/month, once monthly partitions get too large.
- Partition `air_quality`: at 20× scale it likely stops being a small, low-cardinality lookup table — if the source grows to cover multiple states/counties or more pollutants, partition by `date_gmt` (and `state_code / county_code` if the data becomes multi-region) to keep it from becoming one giant unpartitioned table.
- Adjust `weather` partition by year (maybe even month) if the per-day granularity produces too many small partitions relative to the larger data volume.

Task 3. Build a Generic Ingestion Framework

Which components are generic and reusable across all datasets?

The date and time columns are generic. The location, after some processing, can be generic and reusable across all datasets.

Which components remain dataset-specific, and why?

- **For air quality:** Datum, parameter name, units of measure, and method name.
- **For the trip data:** The rating and fare amounts.
- **For the taxi_zone_lookup:** Service zone.
- **For the weather:** Sources of temperature, humidity, wind, etc.

How are transformation rules defined and maintained?

- **General rules:** `readDataGeneric` and `modify_data_generic`, which are reusable for CSV and Parquet files. The modification of the data is done in order to drop rows with bad values (PK is null, the value of a column is negative although it should be positive, timestamps are not well formatted). At the same time, keeping the naming convention to `snake_case`.
- **Dataset-specific rules:** They create the surrogate key and they modify the date and time defined in each dataframe in order to generalize it inside a new field.
- **Maintaining:** Whenever a new dataset is added, generic rules are applied first, then a user needs to define the new dataset-specific rules. In order to be compatible, they would define a datetime column and a PK column. The types of data for each dataset are defined in the schema. If a new column is added, or a type is modified, then the schema needs to be modified as well.

How are metadata (e.g., schema versions, ingestion statistics) managed?

The metadata is collected through the pipeline function. It defines manually the schema version, and the time and the counting of the records is done inside. `execute_pipeline` emits, per dataset: `source_records` (rows read from the file), `out_of_scope_records` (rows removed by an optional `scope_func`), `processed_records` (rows actually put through the quality checks), `rejected_records` (rows those checks dropped), `final_clean_records`, and `execution_time_seconds`. Scope and rejection are reported separately on purpose — air quality discards 8,087,666 rows as out-of-scope simply because they describe other parts of the country, and folding those into `rejected_records` would make a clean dataset look like a broken one.

How does your design reduce code duplication and simplify future maintenance?

The generic function for reading and applying transformations reduces code duplication. At the same time, the pipeline function is doing all the metadata fetching (processing time, number of records before and after processing, version).

If the municipality adds 20 new datasets next year, what changes would be required to your framework?

Each new dataset would require a dedicated function to define the primary key as well as a timestamp/location column. Other than that, the pipeline would take the dataframe through all required stages.

Task 4. Design a Common Data Model

A standard timestamp format The standard timestamp format was created for 3/4 datasets. For the weather data, we appended the year, month, and day to the hour and minute. For the air quality, we

appended the date with the time formats. The trip data already had the timestamp, but we extracted the year and month for partitioning.

Consistent naming conventions We decided on the snake format for database columns (hence modified all the names to lowercase, with "_" instead of " ").

Rules for handling missing values For the weather missing values (primarily in the snow category), the schema allows having null values. Whenever the timestamp data or the primary key data was missing, we decided to eliminate the row as it could not correlate to the other datasets. For the taxi trips, we required `passenger_count` and `trip_distance` to be positive. Fare columns are not checked, so 104,902 rows with a negative `fare_amount` remain in `trip_data`.

Common data types The data types are defined in the schema for each dataset. Hence, all datasets have common data types (Long, Integer, String, Double, Timestamp).

Document every transformation. See section "How are transformation rules defined and maintained?" for more details.

Task 5. Build the Integration Pipeline

The integration pipeline (`build_integrated_trips`) enriches every cleaned trip with weather, air quality and pickup/dropoff zone information, writing `output_data/integrated_taxi_trips` partitioned by year and month. Trips are the anchor entity: every join is a left join from `trip_data`, so enrichment can only add columns, never remove or duplicate a trip — verified by comparing `trip_clean.count()` with the integrated row count.

How should an hourly weather observation be associated with a taxi trip? Each trip's `tepep_pickup_datetime` is truncated down to the hour and joined on equality to the weather table's standardized timestamp (see Task 4). Weather is recorded hourly, so this is a clean 1-to-1 lookup: the trip inherits temperature, humidity, precipitation, snow depth, wind speed, pressure and condition code. Truncating *down* deliberately picks the observation at or before pickup (a 14:45 pickup uses the 14:00 reading) rather than the nearest one — it is deterministic and avoids look-ahead. The source is a single station (8,784 rows = 366 days × 24 hours), so it is a citywide proxy by construction.

How should hourly air-quality measurements be associated with a taxi trip? This requires a spatial decision before the temporal one. The raw EPA file is a *nationwide* extract — 8,139,551 rows from 925 monitoring sites across 53 distinct `state_code` values, of which New York City is only 0.64%. Associating trips by hour alone would blend Californian readings into a New York taxi trip, and would silently mix time zones, since `timestamp_local` is Eastern in New York, Pacific in California and Atlantic in Puerto Rico. `air_quality_scope` therefore filters to the New York City boroughs (`state_code` = 36, `county_code` in 005/047/081) before any cleaning, after which `timestamp_local` is unambiguously Eastern and directly comparable to the trip timestamps. Scoping is deliberately a separate step from `air_quality_process`: the 8,087,666 discarded rows are valid measurements about other regions, so `execute_pipeline` reports them as `out_of_scope_records` rather than data-quality `rejected_records`.

The temporal association mirrors the weather strategy: the filtered data is truncated to `aq_hour`, grouped, and pivoted on `parameter_name` with `avg(sample_measurement)` — one column per pollutant, each keeping its own unit rather than being averaged across parameters. This yields exactly one row per hour, so the left join onto `pickup_hour` cannot fan out. PM2.5 (`parameter_code` = 88101) is the only pollutant in this extract, so `pm25` is currently the sole air-quality column.

We deliberately use a **citywide NYC hourly average** rather than matching each trip to its own borough's station: only Bronx, Brooklyn and Queens have any PM2.5 station (2 sites each), and those are precisely the boroughs that barely generate trips. Of the 9,554,778 raw Q1 pickups, Manhattan alone accounts for **89.55%** and has no station, while the three boroughs that do have stations supply only **10.05%** between them:

Pickup borough	Trips	Share	PM2.5 station?
Manhattan	8,556,766	89.55%	no
Queens	837,886	8.77%	yes
Brooklyn	97,219	1.02%	yes
Bronx	24,959	0.26%	yes
Staten Island	234	0.00%	no

The remaining 0.40% (37,714 pickups) are EWR, zones marked unknown, and location IDs absent from the lookup. Borough-level matching would be false precision for roughly nine trips in ten while adding join complexity, so a citywide average is the honest representation of what these six stations can measure.

How should missing observations be handled? All four enrichment joins are `left` joins, so an absent weather record, air-quality record or zone leaves the corresponding columns `null` rather than dropping the trip — consistent with the Task 4 rule that only rows missing their *own* primary key or timestamp are eliminated. Values are deliberately **not** imputed: a null truthfully records "no observation", whereas an imputed one would be indistinguishable from a real measurement downstream. In practice the gaps are negligible. The NYC stations cover all 2,184 hours of January–March 2024, and exactly **18 of 8,480,540 rows (0.0002%)** have a null `pm25` — the same 18 that have a null `temp`. All 18 are trips whose own `tpep_pickup_datetime` is corrupt (2002, 2008, 2009 and 2023, visible as stray partitions in `trip_data`) and therefore lie outside every coverage window; `pickup_borough` and `dropoff_borough` have no nulls at all.

What are the limitations of your integration strategy?

- **Hourly granularity:** trips starting in the same hour all receive identical conditions, so those near an hour boundary lose precision.
- **Sparse sensor network, citywide averaging:** six stations in three boroughs averaged into one hourly figure, while Manhattan — 89.55% of pickups — has none, so for ~90% of trips the value is measured in a different borough. Weather is citywide for a different reason: it comes from one station.
- **Coarse spatial resolution:** enrichment is keyed on pickup hour and, for zones, on `LocationID`; the weather and air-quality values are never matched to where the trip actually was, so a rain cell over one borough or pollution on a single corridor is invisible.
- **Pickup-time only:** conditions at drop-off, which may differ meaningfully on longer trips, are not captured.
- **Single pollutant in practice:** the pivot is generic across pollutants, but this extract contains only PM2.5, and no composite index (e.g. an AQI) is computed.
- **Timezone handling:** `timestamp_local` is comparable to the trip timestamps only because every retained station is Eastern — the filter, not an explicit timezone conversion, is what makes this hold. Extending the pipeline to another city would require revisiting this rather than simply widening the filter.